

Image Processing/Vision Course Project

- **Objective:** evaluate the **robustness** of image processing and vision algorithms/models
 - Select **a public dataset:** KITTI,...,
 - Select **3 distortions**
 - Select **3 tasks:** Corner-detection, Object Detection, ...
 - Select a **model/algorithm** for each task
- For each task/dataset combination:
 - **Baseline:** measure performance on clean images
 - **Distortion:** apply distortions (noise, rain, low-light) and measure degradation of methods
 - **Improvement:** Try to measure **two** approaches
 - **Enhance** distorted images during pre-processing (denoise, de-rain, ...)
 - **Fine-tune models** (for DL based methods)

Example

- Select dataset: KITTI
- Distortion
 - Dark images
 - Rain
 - Speckle Noise
- Algorithms
 - Edge detection
 - Object detection
 - Keypoint detection

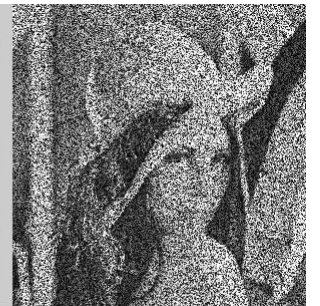
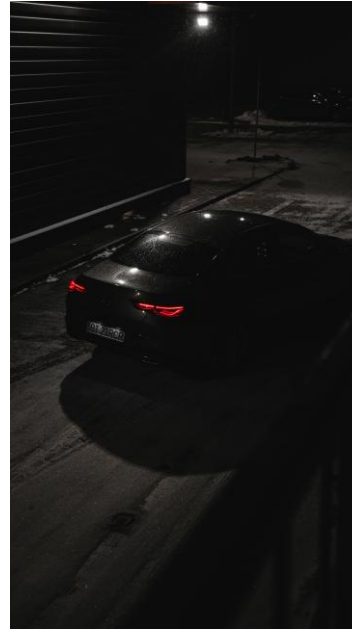
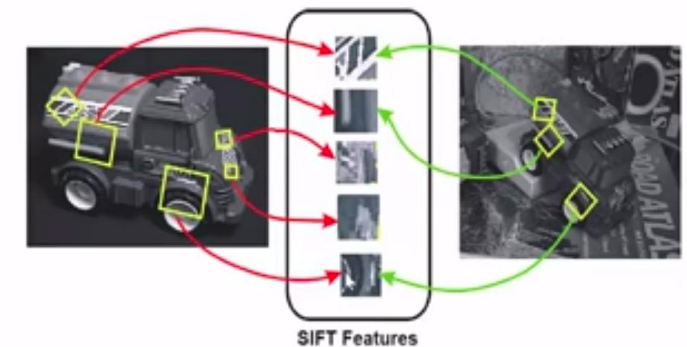
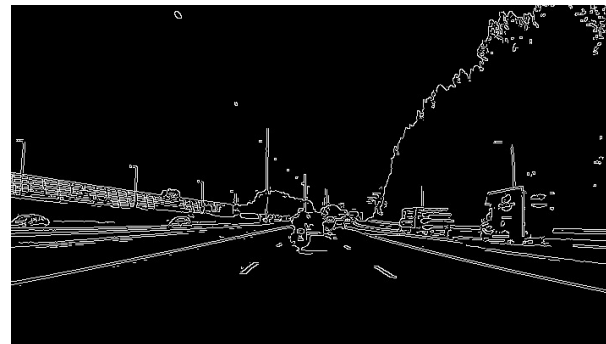


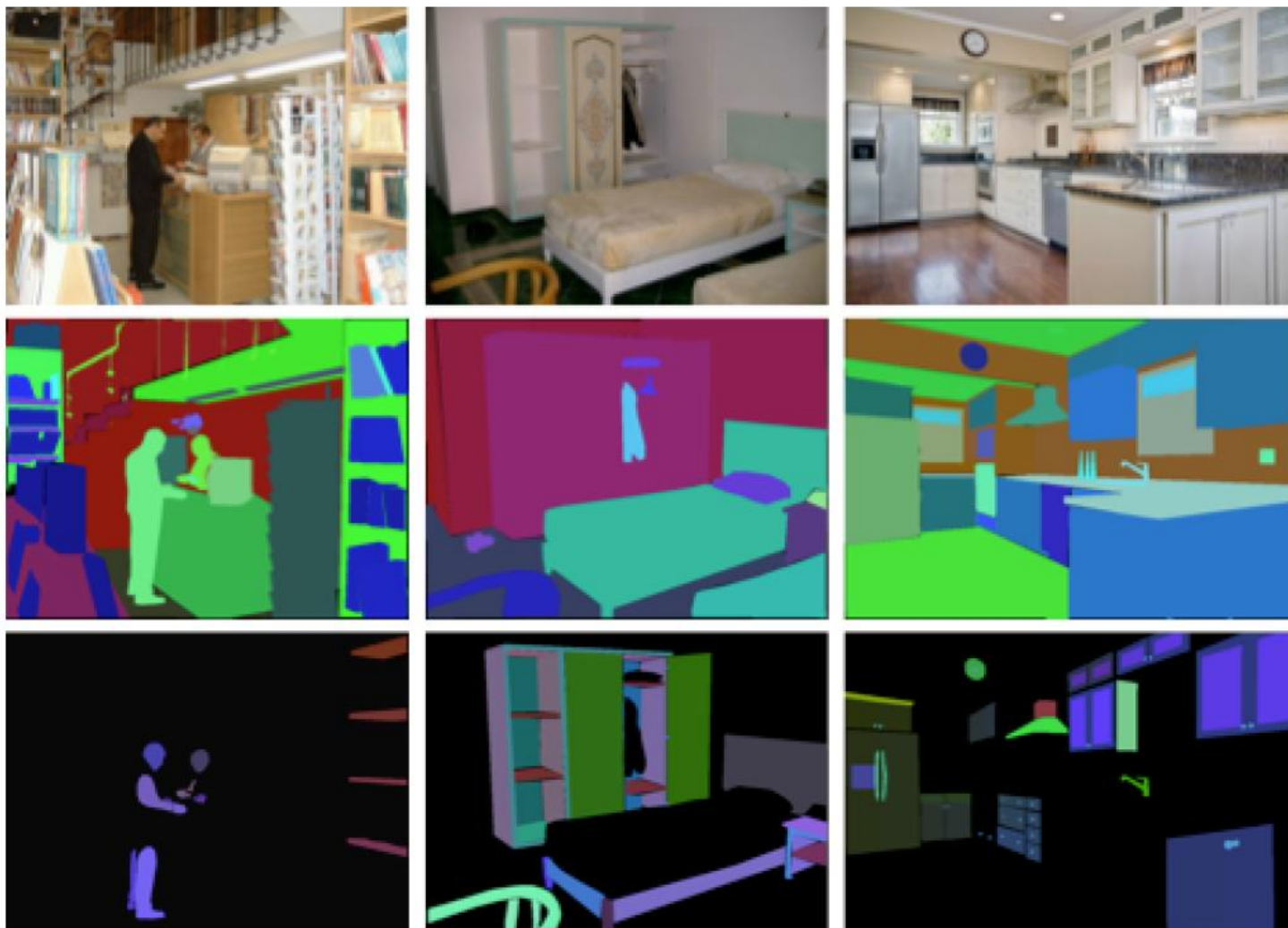
Fig 1: Original Image

Speckle Noisy Image



Part 1: Measure on Clean Images

Example: ADE2K dataset



1. Prepare Dataset and a sample

```
random.seed(7)

ds = load_dataset("nateraw/ade20k-tiny", split="train")
N = len(ds)

idxs = random.sample(range(N), 4)
samples = [ds[i] for i in idxs]

images = [s["image"] for s in samples]
masks = [s["label"] for s in samples]
```

Plot sample with labels

```
def overlay_mask(img_pil, mask_pil, alpha=0.45):
    img = np.array(img_pil.convert("RGB")).astype(np.float32)
    m = np.array(mask_pil).astype(np.int32)
    rng = np.random.default_rng(0)
    palette = rng.integers(0, 255, size=(256, 3), dtype=np.uint8)
    color = palette[(m % 256).astype(np.int32)]
    out = (img * (1 - alpha) + color.astype(np.float32) * alpha).clip(0, 255).astype(np.uint8)
    return Image.fromarray(out)

fig, axes = plt.subplots(1, 4, figsize=(20, 5))
for ax, img, m in zip(axes, images, masks):
    ax.imshow(overlay_mask(img, m))
    ax.axis("off")
plt.tight_layout()
plt.show()
```

Result



Run ORB feature detector



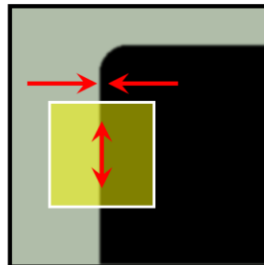
```
def orb_overlay(img_pil, nfeatures=800):  
    img = np.array(img_pil.convert("RGB"))  
    gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)  
    orb = cv2.ORB_create(nfeatures=nfeatures)  
    kps = orb.detect(gray, None)  
    out = cv2.drawKeypoints(img, kps, None, flags  
    return out, kps
```

```
fig, axes = plt.subplots(1, 4, figsize=(20, 5))  
for ax, img in zip(axes, images):  
    out, kps = orb_overlay(img)  
    ax.imshow(out)  
    ax.set_title(f"ORB keypoints: {len(kps)}")  
    ax.axis("off")  
plt.tight_layout()  
plt.show()
```

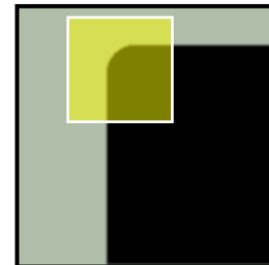
Corner Detection: Basic idea



"flat" region:
no change in
all directions



"edge" : no change
along the edge
direction



"corner" : significant
change in all directions
with small shift

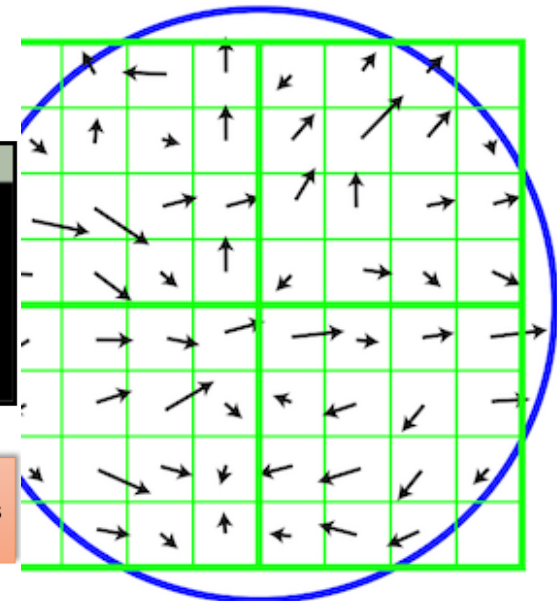


Image gradients

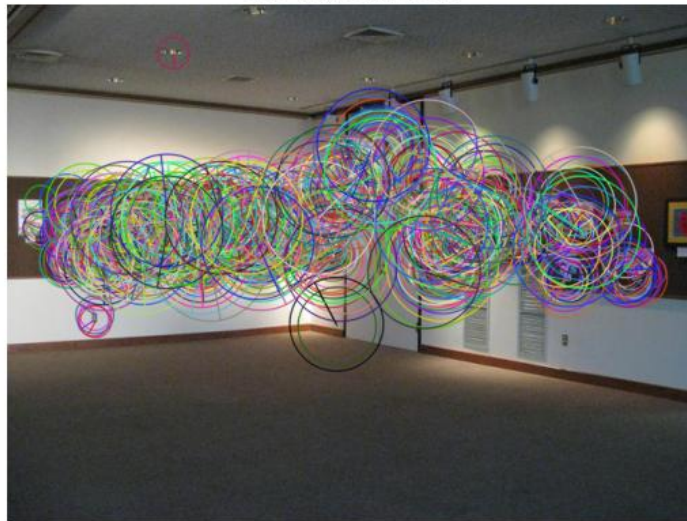


Results

ORB keypoints: 761



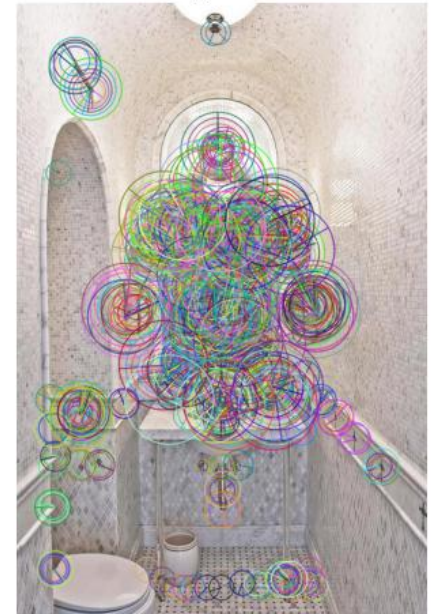
ORB keypoints: 800



ORB keypoints: 800



ORB keypoints: 800



Run Pretrained Object Detection

```
model = YOLO("yolov8n.pt")

def yolo_overlay(img_pil, conf=0.25):
    img = np.array(img_pil.convert("RGB"))
    r = model.predict(img, conf=conf, verbose=False)[0]
    out = r.plot()
    return out, r

fig, axes = plt.subplots(1, 4, figsize=(20, 5))
for ax, img in zip(axes, images):
    out, r = yolo_overlay(img)
    ax.imshow(out)
    ax.set_title(f"YOLO detections: {len(r.bboxes)}")
    ax.axis("off")
plt.tight_layout()
plt.show()
```

Results

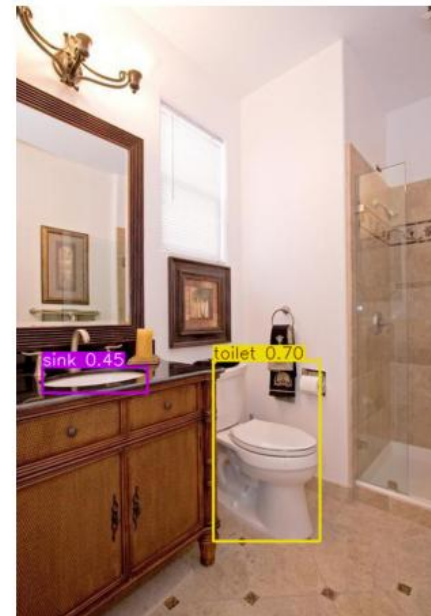
YOLO detections: 1



YOLO detections: 0



YOLO detections: 2



YOLO detections: 5



Run Pretrained Segmentation

```
seg_processor = SegformerImageProcessor.from_pretrained("nvidia/segformer-b0-finetuned-ade-512-512")
seg_model     = SegformerForSemanticSegmentation.from_pretrained("nvidia/segformer-b0-finetuned-ade-512-512")
seg_model.eval()

def predict(img_pil):
    inputs = seg_processor(images=img_pil, return_tensors="pt")
    with torch.no_grad():
        logits = seg_model(**inputs).logits
    up = F.interpolate(logits, size=(img_pil.height, img_pil.width),
                       mode="bilinear", align_corners=False)
    return up.argmax(1).squeeze().numpy().astype(np.int32)  # 0..149
```

Display

```
id2label = seg_model.config.id2label
NUM_CLS = 150
PALETTE = np.random.default_rng(42).integers(40, 230, size=(NUM_CLS, 3), dtype=np.uint8)

def colorize(mask_idx):
    return PALETTE[np.clip(mask_idx, 0, NUM_CLS - 1)]

def seg_overlay(img_rgb, mask_idx, alpha=0.55):
    return (img_rgb * (1 - alpha) + colorize(mask_idx) * alpha).clip(0, 255).astype(np.uint8)

preds = [predict(img) for img in images]
gt_arrs = [np.array(m) for m in masks] # 0..150 (ADE20K raw)

fig, axes = plt.subplots(4, 3, figsize=(14, 16))
for j, title in enumerate(["Clean", "Ground Truth", "Prediction"]):
    axes[0, j].set_title(title, fontsize=12)
for i, img in enumerate(images):
    img_np = np.array(img.convert("RGB"))
    axes[i, 0].imshow(img_np)
    axes[i, 1].imshow(seg_overlay(img_np, np.clip(gt_arrs[i] - 1, 0, NUM_CLS - 1)))
    axes[i, 2].imshow(seg_overlay(img_np, preds[i]))
    for j in range(3):
        axes[i, j].axis("off")
plt.tight_layout()
plt.show()
```

Results

Clean



Ground Truth

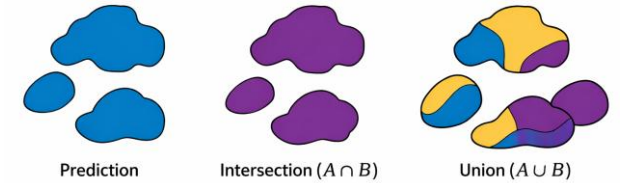


Prediction



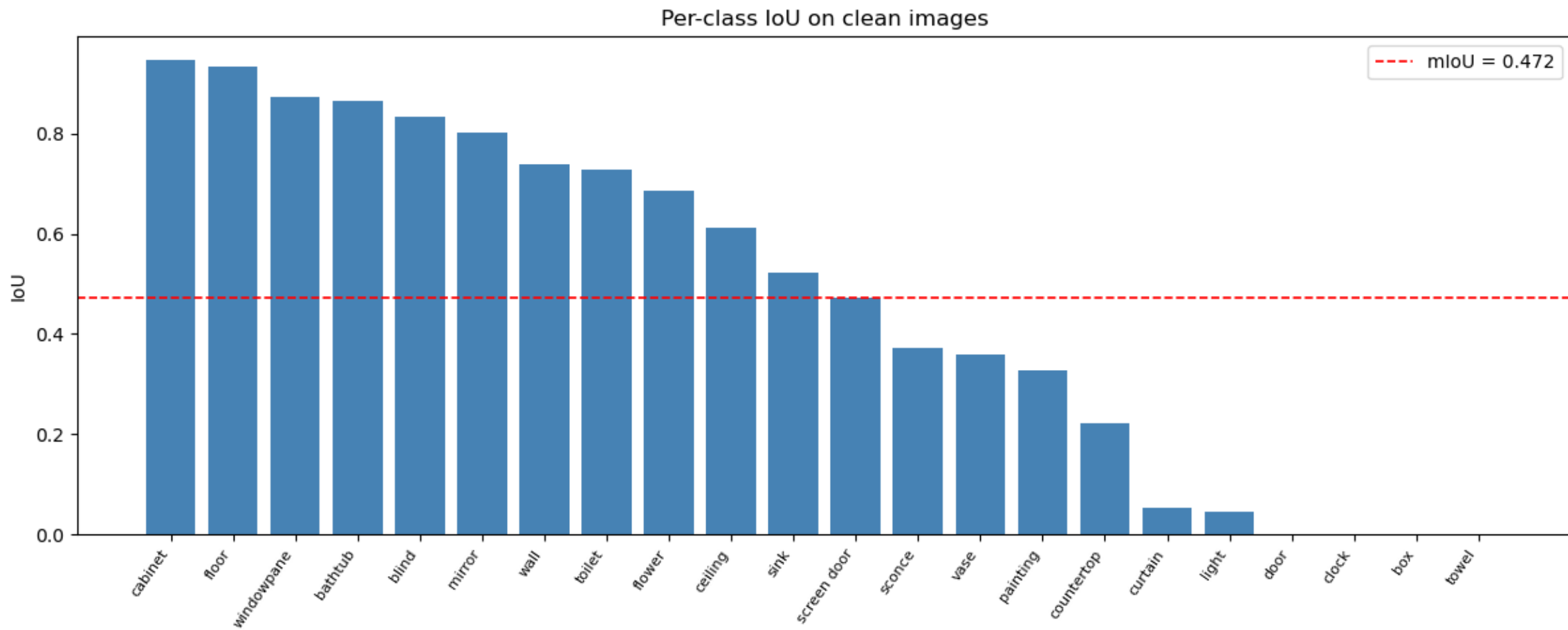
Compute IoU metric

```
def compute_iou(pred_idx, gt_raw):  
    """pred 0..149 | gt 0..150 (ADE20K: 0=unlabeled, 1..150=classes → shift by -1)"""  
    valid = gt_raw > 0  
    gt_0 = gt_raw - 1  
    ious = {}  
    for c in np.unique(gt_0[valid]):  
        p = pred_idx == c  
        g = (gt_0 == c) & valid  
        union = int((p | g).sum())  
        if union:  
            ious[int(c)] = float((p & g).sum()) / union  
    return ious
```



$$IoU = \frac{|A \cap B|}{|A \cup B|}$$

Results



Part 2: Measure on Distorted Images

Introducing Distortions

```
distortions = {
    "GaussNoise":      albumentations.GaussNoise(var_limit=(500.0, 1500.0), p=1.0),
    "SevereJPEG":      albumentations.ImageCompression(quality_lower=1, quality_upper=5, p=1.0),
    "LowLight":        albumentations.RandomBrightnessContrast(brightness_limit=(-0.8, -0.6), contrast_limit=(0.0, 0.0), p=1.0),
}

def apply_aug(img_pil, aug):
    img = np.array(img_pil.convert("RGB"))
    out = aug(image=img)["image"]
    return out

fig, axes = plt.subplots(4, len(distortions), figsize=(14, 14))
for row, img in enumerate(images):
    for col, (name, aug) in enumerate(distortions.items()):
        axes[row, col].imshow(apply_aug(img, aug))
        axes[row, col].axis("off")
        if row == 0:
            axes[row, col].set_title(name)
plt.tight_layout()
plt.show()
```

Results

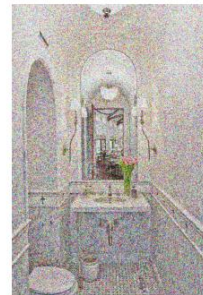
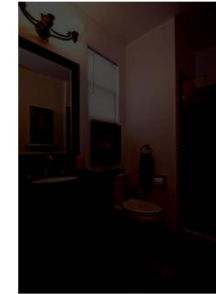
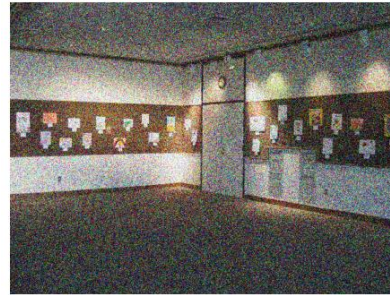
GaussNoise



SevereJPEG



LowLight



Features on Noisy Images

Distortion: GaussNoise

Clean ORB (761)



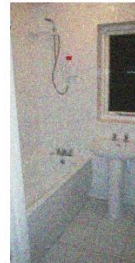
Dist ORB (800)



Clean YOLO (1)



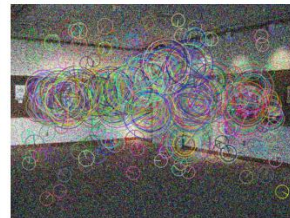
Dist YOLO (0)



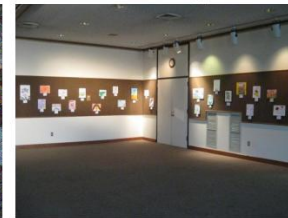
Clean ORB (800)



Dist ORB (800)



Clean YOLO (0)



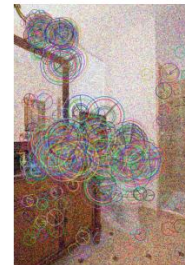
Dist YOLO (0)



Clean ORB (800)



Dist ORB (800)



Clean YOLO (2)



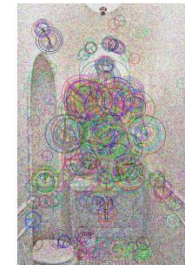
Dist YOLO (0)



Clean ORB (800)



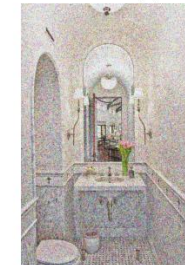
Dist ORB (800)



Clean YOLO (5)



Dist YOLO (0)



Dark Images

Distortion: LowLight

Clean ORB (761)



Dist ORB (91)



Clean YOLO (1)



Dist YOLO (0)



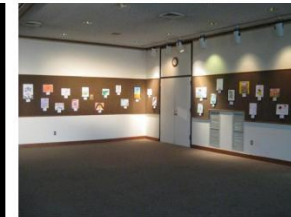
Clean ORB (800)



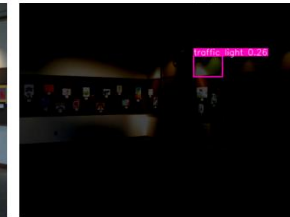
Dist ORB (681)



Clean YOLO (0)



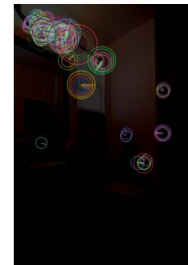
Dist YOLO (1)



Clean ORB (800)



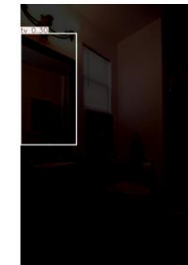
Dist ORB (153)



Clean YOLO (2)



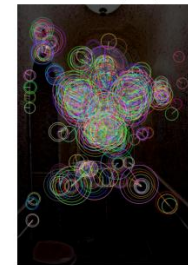
Dist YOLO (1)



Clean ORB (800)



Dist ORB (734)



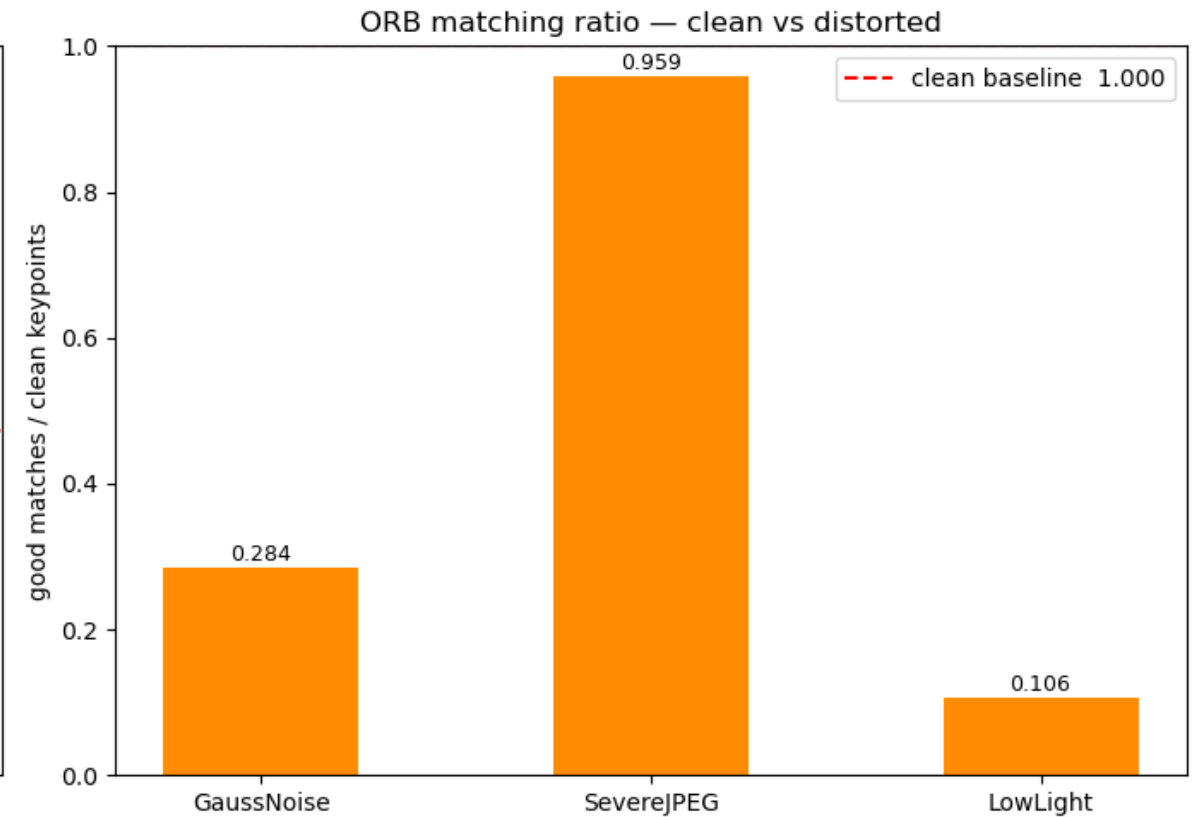
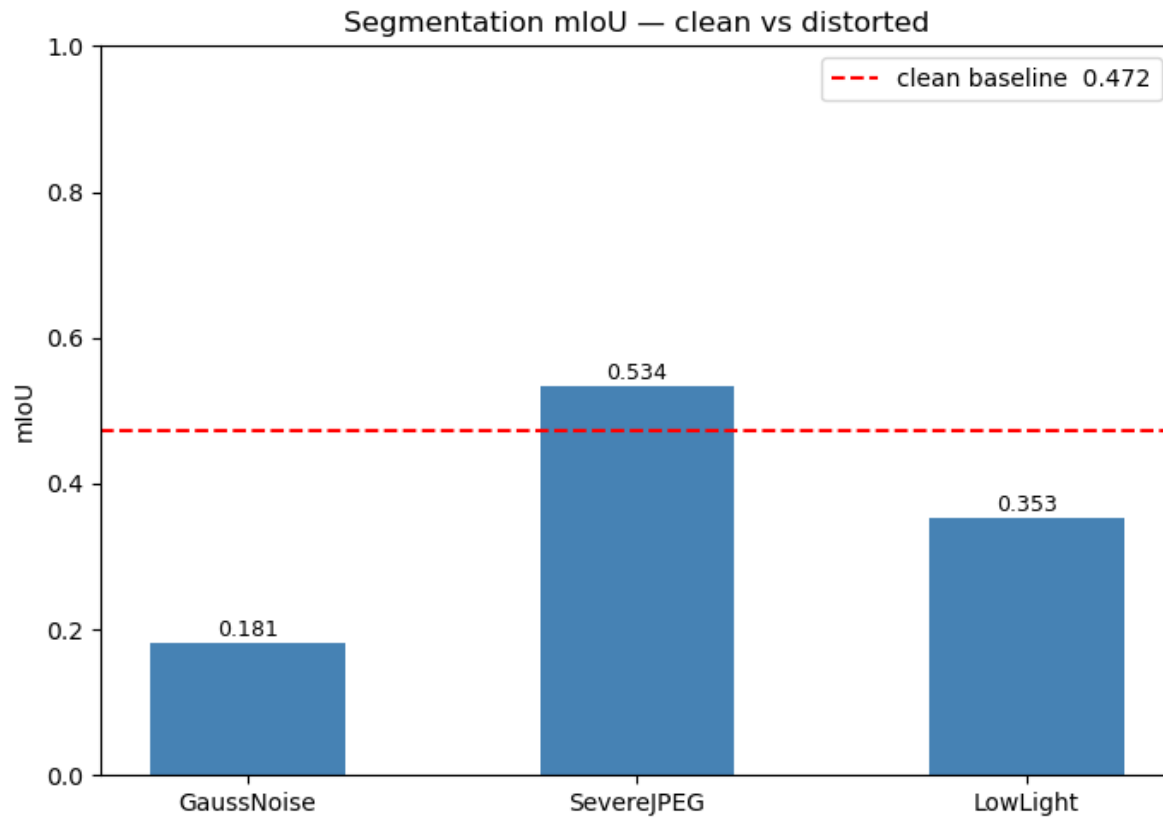
Clean YOLO (5)



Dist YOLO (2)



Measuring Feature Matching Accuracy



degradation level -> SNR

```
def compute_snr(clean_rgb, dark_rgb):  
    """SNR (dB) = 10 * log10(signal_power / noise_power), noise = clean - dark."""  
    clean      = clean_rgb.astype(np.float64)  
    noise      = clean - dark_rgb.astype(np.float64)  
    signal_power = np.mean(clean ** 2)  
    noise_power  = np.mean(noise ** 2)  
    return 10.0 * np.log10(signal_power / noise_power) if noise_power > 0 else np.inf
```

Signal-to-Noise ratio

- SNR: ratio between signal power and noise power
- Definition:

$$SNR = \frac{P_{signal}}{P_{noise}} = \left(\frac{A_{signal}}{A_{noise}} \right)^2$$

$$SNR(dB) = 10 \log_{10} \left(\frac{P_{signal}}{P_{noise}} \right) = 20 \log_{10} \left(\frac{A_{signal}}{A_{noise}} \right)$$

Performance per SNR

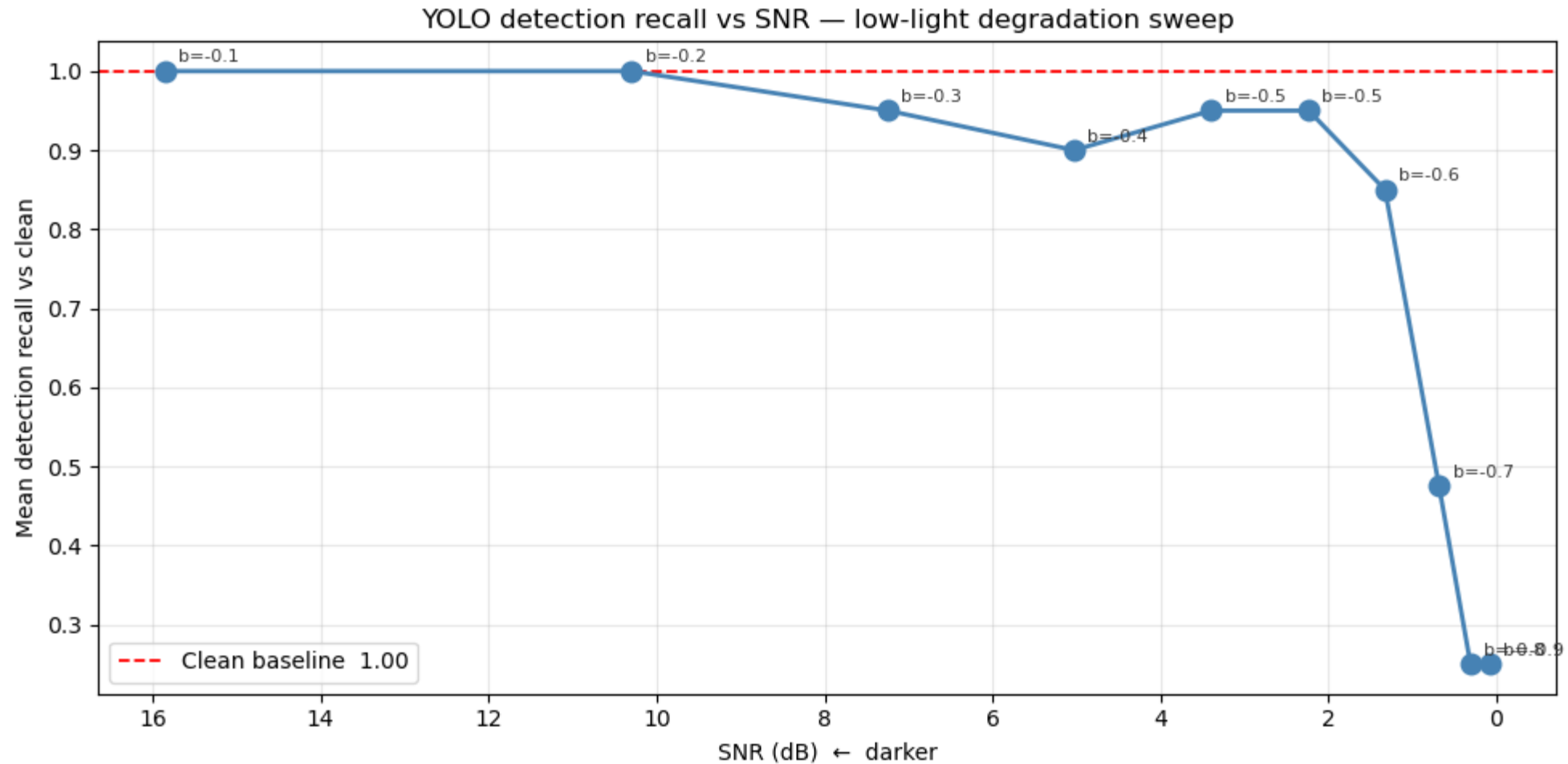
```
✓ for brightness in brightness_levels:
    lvl_snr, lvl_rec = [], []
    ✓ for i, img in enumerate(images):
        clean_rgb = np.array(img.convert("RGB"))
        dark_rgb = darken(img, brightness)

        lvl_snr.append(compute_snr(clean_rgb, dark_rgb))

        ref = clean_ref_boxes[i]
        r = model.predict(dark_rgb, conf=0.25, verbose=False)[0]
        boxes = r.bboxes.xyxy.cpu().numpy() if r.bboxes is not None else np.zeros((0, 4))
        lvl_rec.append(detection_recall(ref, boxes))

    snr_vals.append(float(np.mean(lvl_snr)))
    recall_vals.append(float(np.mean(lvl_rec)))
```

Performance per SNR



Part 3: Measure on Restored Images

Enhancing Images

```
def restore_noise(img_rgb):  
    bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)  
    # Strong NLM denoising (h=25 handles heavy noise), larger search window  
    den = cv2.fastNlMeansDenoisingColored(bgr, None, 25, 25, 7, 35)  
    # Bilateral pass to smooth residual grain while preserving edges  
    den = cv2.bilateralFilter(den, d=9, sigmaColor=80, sigmaSpace=80)  
    out = cv2.cvtColor(den, cv2.COLOR_BGR2RGB)  
    return out
```

```
def restore_jpeg(img_rgb):  
    ycrb = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2YCrCb)  
    y, cr, cb = cv2.split(ycrb)  
    y = cv2.bilateralFilter(y, d=7, sigmaColor=40, sigmaSpace=40)  
    out = cv2.cvtColor(cv2.merge([y, cr, cb]), cv2.COLOR_YCrCb2RGB)  
    return out
```

```
def restore_lowlight(img_rgb):  
    # Gamma correction lifts very dark pixels before local equalization  
    gamma = 0.35  
    lut = (np.arange(256) / 255.0) ** gamma * 255  
    lut = np.clip(lut, 0, 255).astype(np.uint8)  
    img_gamma = cv2.LUT(img_rgb, lut)  
    # Strong CLAHE on L channel for local contrast boost  
    lab = cv2.cvtColor(img_gamma, cv2.COLOR_RGB2LAB)  
    l, a, b = cv2.split(lab)  
    clahe = cv2.createCLAHE(clipLimit=6.0, tileGridSize=(8, 8))  
    l = clahe.apply(l)  
    out = cv2.cvtColor(cv2.merge([l, a, b]), cv2.COLOR_LAB2RGB)  
    return out
```

Results: Noise

Clean



Distorted



Restored



Clean



Distorted



Restored



Results: Low Light

Clean



Distorted



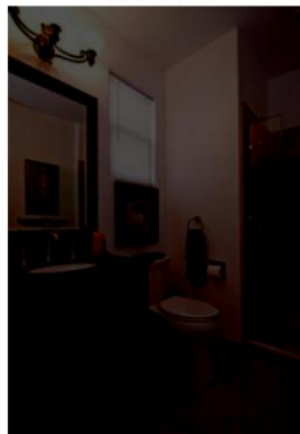
Restored



Clean



Distorted



Restored

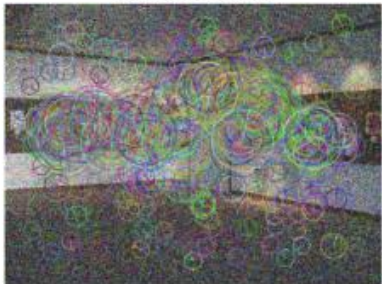


Run on enhanced images

Clean ORB (800)



Dist ORB (800)



Enh ORB (800)



Clean YOLO (0)



Dist YOLO (0)



Enh YOLO (0)



Clean ORB (800)



Dist ORB (800)



Enh ORB (800)



Clean YOLO (2)



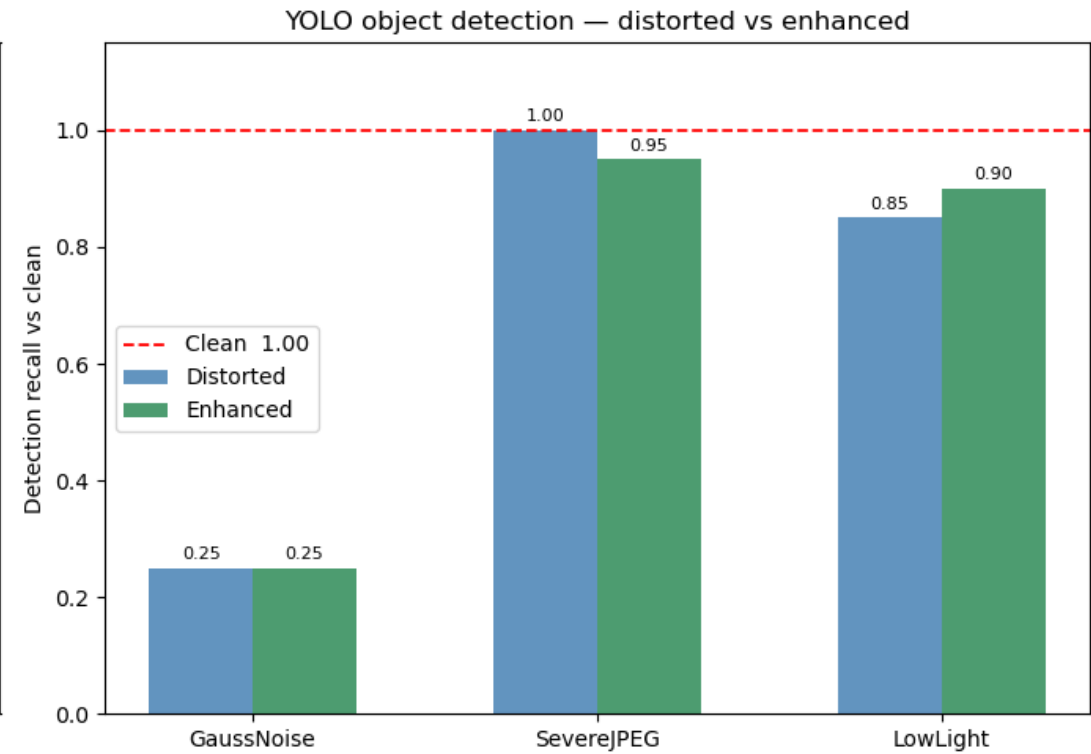
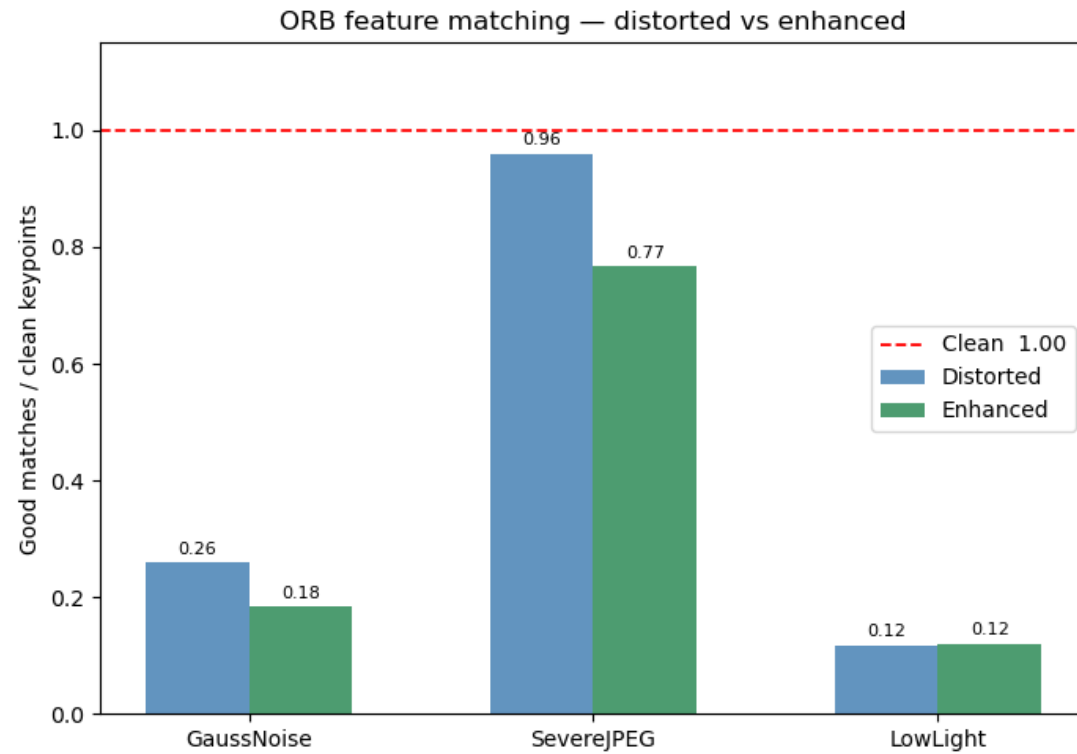
Dist YOLO (0)



Enh YOLO (1)



Comparing results



Part 4: Fine-Tuning Models on Distorted Images

Fin-Tuning Model: Create Labels from Clean

```
0 0.512344 0.601250 0.318750 0.427500  
2 0.245000 0.540833 0.120000 0.186667
```

```
def save_yolo_label(txt_path, boxes_xyxy, cls_ids, w, h):  
    lines = []  
    for (x1, y1, x2, y2), c in zip(boxes_xyxy, cls_ids):  
        cx = ((x1 + x2) / 2) / w  
        cy = ((y1 + y2) / 2) / h  
        bw = (x2 - x1) / w  
        bh = (y2 - y1) / h  
        lines.append(f"{int(c)} {cx:.6f} {cy:.6f} {bw:.6f} {bh:.6f}")  
    txt_path.write_text("\n".join(lines))
```

```
pseudo_names = []  
for i, img_pil in enumerate(images):  
    img = np.array(img_pil.convert("RGB"))  
    h, w = img.shape[:2]  
    r = model.predict(img, conf=0.35, iou=0.5, verbose=False)[0]  
    boxes = r.boxes  
    if boxes is None or len(boxes) == 0:  
        continue  
    xyxy = boxes.xyxy.cpu().numpy()  
    cls = boxes.cls.cpu().numpy()  
    img_path = work/"images/train"/f"im_{i}.jpg"  
    lbl_path = work/"labels/train"/f"im_{i}.txt"  
    import cv2  
    cv2.imwrite(str(img_path), cv2.cvtColor(img, cv2.COLOR_RGB2BGR))  
    save_yolo_label(lbl_path, xyxy, cls, w, h)  
    pseudo_names.append(f"im_{i}.jpg")
```

Fine-Tune YOLO

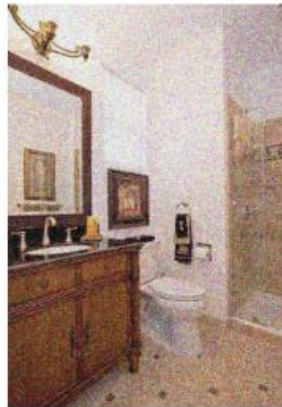
```
ft_model = YOLO("yolov8n.pt")
_ = ft_model.train(data=str(work/"data.yaml"), imgsz=640, epochs=3, batch=2, device="cpu", verbose=False)

best = Path(ft_model.trainer.best) if hasattr(ft_model, "trainer") else (work/"runs/detect/train/weights/best.pt")
ft = YOLO(str(best))
```

Clean (pre) 2



Dist (pre) 0



Enh (pre) 1



Dist (ft) 2



Project Choices

1

Select a dataset

- ade20K

2

Select 3 vision tasks

- Feature detection
- Semantic segmentation (with GT)
- Object Detection

3

Select evaluation metrics

- IoU: segmentation and object detection
- Match accuracy for feature detection

4

Select method for each task

- ORB for feature detection
- SegFormer for semantic segmentation
- YOLOn8 for object detection

5

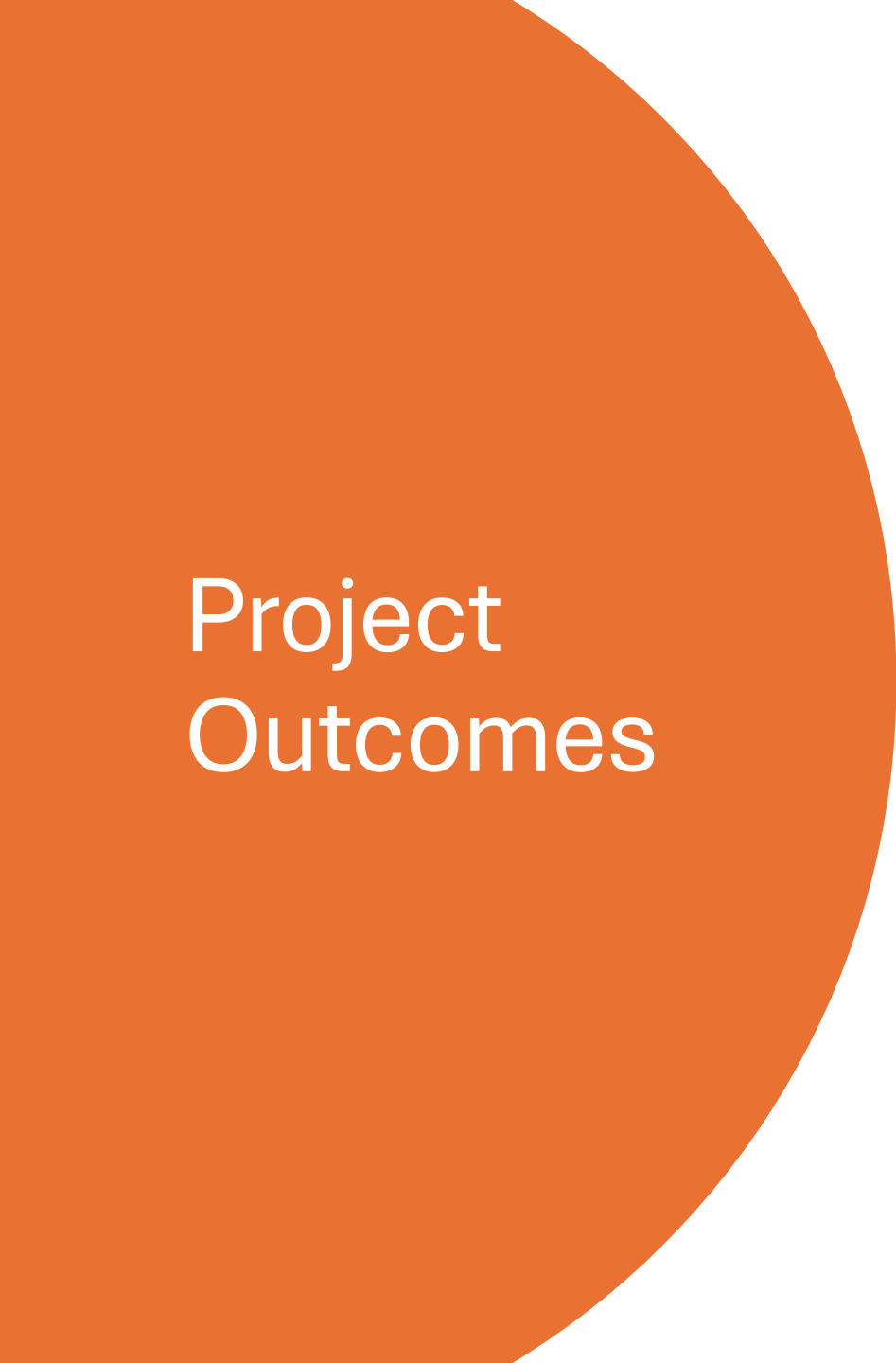
Select 3 distortions

- Noise
- Compression
- Low Light

6

Select enhancements for each distortion

- Noise: Non-Local Means
- Compression: Interpolation with Bilateral Filtering
- Low Lights: Contrast Limited Adaptive Histogram Equalization

A large orange circle is positioned on the left side of the slide, partially cut off by the edge. The text 'Project Outcomes' is written in white inside this circle.

Project Outcomes

Baseline performance (vs. GT),
use as GT for following steps

Performance on distorted
images (per distortion/class)

Performance on enhanced
images (per distortion/class)

Performance of fine-tuned
models (on distorted images)

This PPT skips (but your project must have)



Skip Steps

No distortion, all enhancement, all methods



Small scale

Few example evaluation and training



Detailed evaluation

Range of distortion intensities
Show accuracy per intensity
(measure as SNR)

Advices



Start early, plan, parallelize, and make incremental progress

See the suggested plan, update artifacts in Git



Find interesting data, methods, models, and handle uncertainty

Consult with ChatGPT for datasets, tasks, methods, distortions, enhancements, metrics



Don't write/ generate long code, use AI and libraries

Most of the standard tasks had been implemented elsewhere



Avoid lock-in

Many models/method, metrics for each task, many libraries, and different ways to implement



Adapt on the fly

Make decisions with the information you have (e.g. methods), prepare to change decisions

Submission

Register in the course project register

- Link is on the course Moodle page
- Team members: names and email
- GitHub Repo URL (make sure accessible)

Your repo is your submission

- Detailed ReadMe is your report
- Provide code/data/visuals
- Final PPT for project presentation as an easy-to-read version of ReadMe

Requirements

- A dataset with GT for at **least one** task
- At least **3** different tasks: include low-level and high-level tasks
- At **least one** DL model
- At least **3** distortion methods
- Measure performance: **per class, per SNR**
- Document (in your ReadMe)
 - Your choices
 - Tables of results/metrics
 - Visualization
 - Input/Output processing steps: Image with annotation, before/after
 - Measurements: plot bar plot, curves, comparisons

Suggested Weekly Plan

Week	Task	Artifact
1	Form team, open Git , register	Opened GitHub repo, entry in course project table
2	Research and select dataset, distortions, tasks	A table with decisions embedded in ReadMe (with links)
3	Research and select methods and enhancements	A table with decisions embedded in ReadMe (with links)
4	Download data, visualize images, and annotations	Download and EDA code, sample image grid with annotations embedded in ReadMe
5	Run methods/models on clean data and save outcomes	Folder with outcomes/labels in repo
6	Measure performance of selected method using GT	Table of results, visualization per class
7	Apply distortions and save data	Distortion code, data folder with distorted images, visualization of before/after
8	Run models and measure degradation in performance	Code of model application, tables of performance, visuals of image annotations and performance comparison
9	Apply enhancements and measure performance	Visuals of images/with annotations, side-by-side grid, visuals of performance comparison
10	Fine-Tune model(s)	Code of fine-tuned models, checkpoint/model weights
11	Measure performance of Fine-Tuned model(s)	Table of results in Git , visualization
12	Review ReadMe so far, improve visualization, text, tables	Improved, rich, informative, detailed ReadMe with all decisions and visualization
13	Prepare and upload PPT, Review Repo	Full repo, Slides (PPT and PDF)